# LAB 2: Detecting Suspicious Login Activity Using Machine Learning

**ENSP355 | AI in Cyber Security Lab | 2026-27** — Total: 50 marks

Problem: numeric login-activity data (failed logins, login hour, new-device flag, downloaded MB) ko clean karke Decision Tree se Normal/Suspicious classify karna, K-Means se cluster karna, aur results interpret karna.

> Colab me chalane se pehle `simple_login_activity.csv` upload karein (left Files panel → Upload).

```
# Setup: CSV load
import pandas as pd
import numpy as np

df = pd.read_csv('simple_login_activity.csv')
df.head()
```

| | record_id | failed_logins | login_hour | new_device | download_mb | |
|---|---|---|---|---|---|---|
| **0** | 1 | 4.0 | 1 | 0 | 164.9 | Susp |
| **1** | 2 | 3.0 | 3 | 1 | 250.6 | Susp |
| **2** | 3 | 3.0 | 10 | 0 | 145.1 | N |
| **3** | 4 | 2.0 | 11 | 0 | 397.8 | N |
| **4** | 5 | 1.0 | 3 | 0 | 409.3 | N |

Next steps: Generate code with df | New interactive sheet

# Q1. Dataset Inspection and Cleaning (5 marks)

**Aim:** input columns aur target label pehchanna, duplicates hatana, missing numeric values bharna.

- Models ko complete, consistent data chahiye; median missing numeric values ke liye robust hai.
- Duplicate rows analysis ko distort karte hain.
- Target label: `Normal` / `Suspicious`; baaki selected columns input features hain.

```
print(df.info())
print('Missing values before cleaning:')
print(df.isna().sum())
print('Duplicate rows:', df.duplicated().sum())

df = df.drop_duplicates().copy()
df['failed_logins'] = df['failed_logins'].fillna(df['failed_logins'
df['download_mb'] = df['download_mb'].fillna(df['download_mb'].medi

print('Missing values after cleaning:', df.isna().sum().sum())
print('Duplicate rows after cleaning:', df.duplicated().sum())
print('Final rows:', len(df))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101 entries, 0 to 100
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   record_id      101 non-null    int64
 1   failed_logins  100 non-null    float64
 2   login_hour     101 non-null    int64
 3   new_device     101 non-null    int64
 4   download_mb    100 non-null    float64
 5   label          101 non-null    object
dtypes: float64(2), int64(3), object(1)
memory usage: 4.9+ KB
None
Missing values before cleaning:
record_id        0
failed_logins    1
login_hour       0
new_device       0
download_mb      1
label            0
dtype: int64
Duplicate rows: 1
Missing values after cleaning: 0
Duplicate rows after cleaning: 0
Final rows: 100
```

## Q2. Training and Testing Split (5 marks)

**Aim:** features/label alag karke 75/25 train-test split banana.

- Training data se model seekhta hai; testing data unseen records pe performance batata hai.
- Stratified split dono sets me class proportion same rakhta hai.
- `random_state=42` se split reproducible hai.

```
from sklearn.model_selection import train_test_split

features = ['failed_logins', 'login_hour', 'new_device', 'download_
X = df[features]
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

print('Training rows:', len(X_train))
print('Testing rows:', len(X_test))
```

```
Training rows: 75
Testing rows: 25
```

## Q3. Decision Tree Training and Evaluation (10 marks)

**Aim:** Decision Tree train karke accuracy aur confusion matrix interpret karna.

- Tree labelled data se if-then rules seekhta hai; `max_depth=4` tree ko readable rakhta hai aur overfitting kam karta hai.
- Confusion matrix: sahi Normal/Suspicious faisle vs false alarms (FP) aur missed attacks (FN).

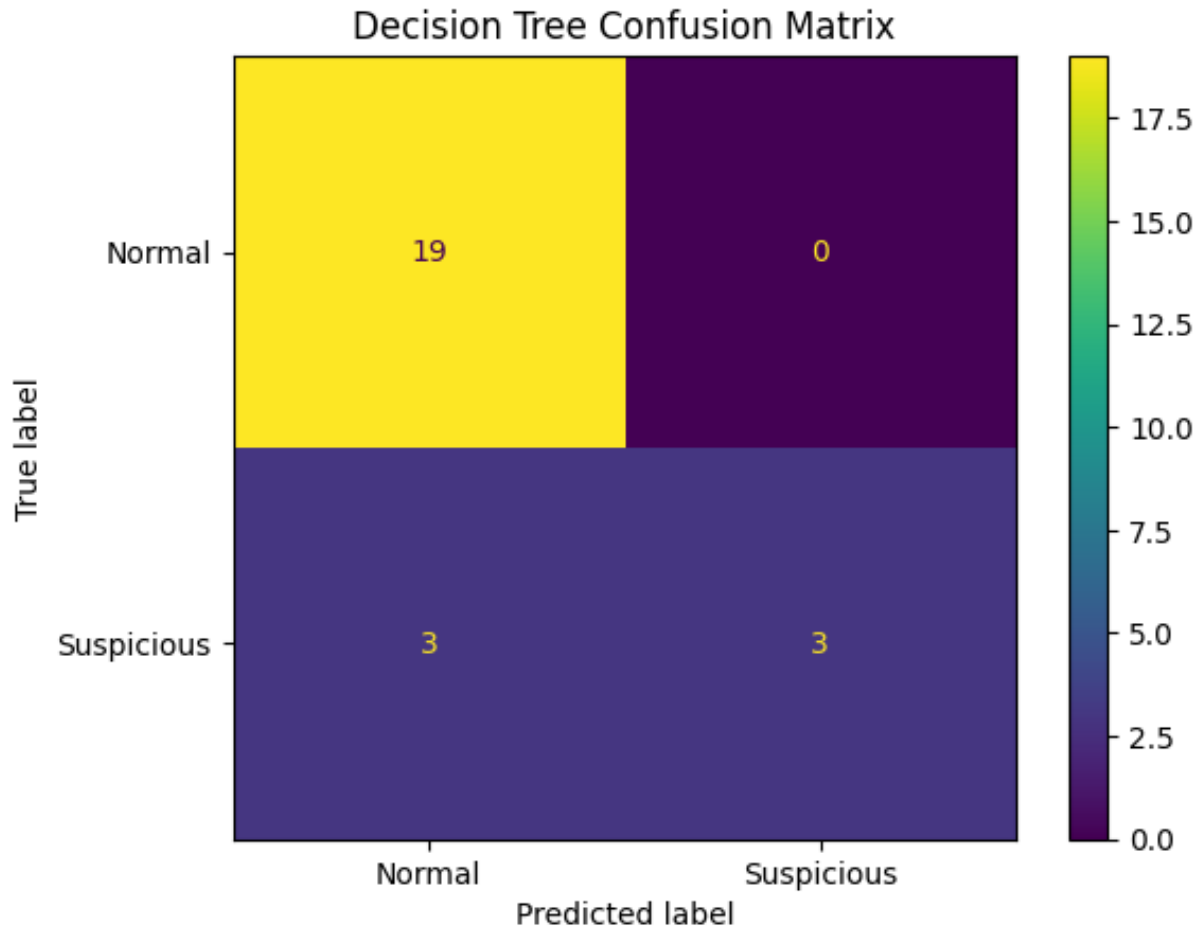
```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay

model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy*100:.2f}%')

ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
    labels=['Normal', 'Suspicious'])
plt.title('Decision Tree Confusion Matrix')
plt.show()
```

```
Accuracy: 88.00%
```



# Q4. K-Means Clustering (10 marks)

**Aim:** unsupervised tareeke se similar login activity group karna aur labels se compare karna.

- K-Means labels use nahi karta, sirf numeric similarity pe group banata hai.
- Alag scales (hour vs MB) ki wajah se features ko standardize karna zaroori hai.
- Clusters ka labels se perfect match hona zaroori nahi.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X_scaled)

cluster_summary = df.groupby('cluster')[features].mean().round(2)
label_summary = pd.crosstab(df['cluster'], df['label'])
print(cluster_summary.to_string())
print()
print(label_summary.to_string())
```

```
         failed_logins  login_hour  new_device  download_mb
cluster
0                 2.00       12.81         1.0       192.69
1                 2.16       11.63         0.0       236.15

label    Normal  Suspicious
cluster
0            24          13
1            51          12
```

# Q5. Feature Importance and New Record Prediction (5 marks)

**Aim:** tree ne kin features ko sabse zyada use kiya, aur 3 naye records predict karna.

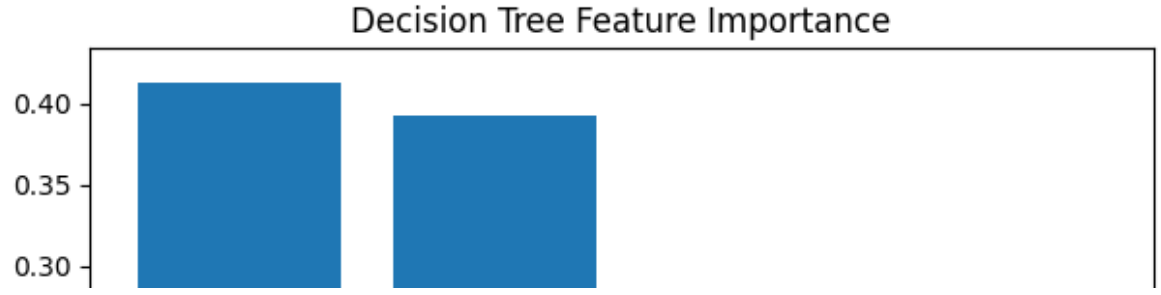
- Feature importance batata hai tree ne split ke liye kis feature ko kitna use kiya.
- Ye model-specific hai; causation prove nahi karta.

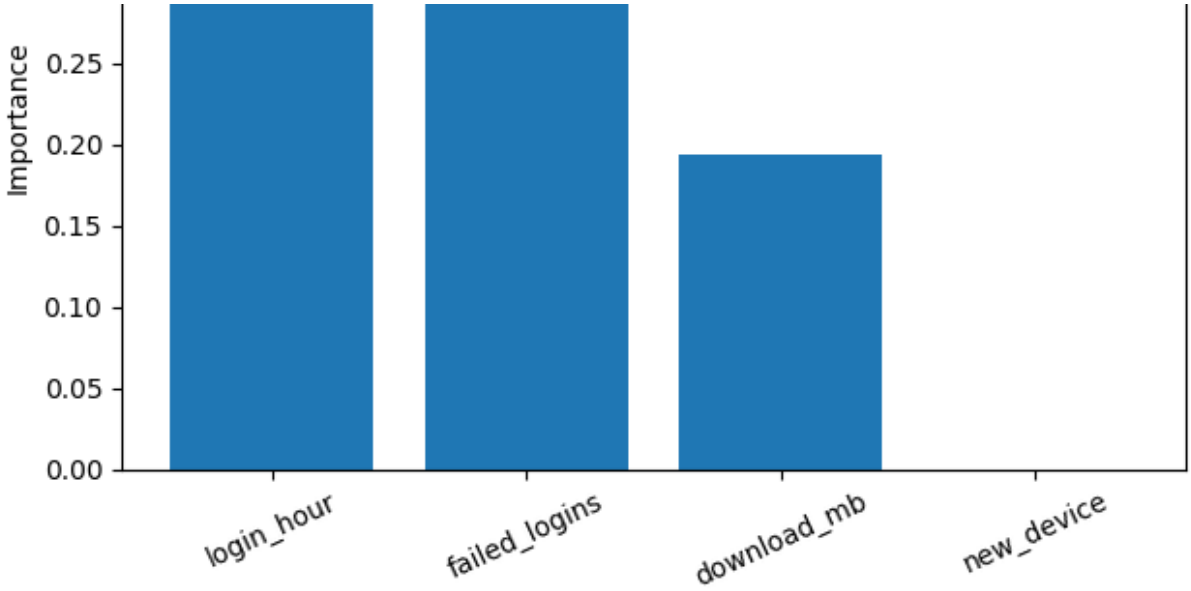
```
importance = pd.DataFrame({
    'feature': features,
    'importance': model.feature_importances_
}).sort_values('importance', ascending=False)
print(importance.to_string(index=False))

plt.bar(importance['feature'], importance['importance'])
plt.title('Decision Tree Feature Importance')
plt.ylabel('Importance')
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()

new_records = pd.DataFrame([
    {'failed_logins': 0, 'login_hour': 10, 'new_device': 0, 'downlo
    {'failed_logins': 8, 'login_hour': 2,  'new_device': 1, 'downlo
    {'failed_logins': 2, 'login_hour': 23, 'new_device': 1, 'downlo
])
new_records['prediction'] = model.predict(new_records)
print(new_records.to_string(index=False))
```

```
      feature  importance
   login_hour    0.413391
failed_logins    0.392826
  download_mb    0.193783
   new_device    0.000000
```






| failed_logins | login_hour | new_device | download_mb | prediction |
|---|---|---|---|---|
| 0 | 10 | 0 | 45 | Normal |
| 8 | 2 | 1 | 180 | Suspicious |
| 2 | 23 | 1 | 900 | Suspicious |

## Q6. False Positives, False Negatives and Reflection (5 marks)

**Aim:** do zaroori model errors samjhana aur model ki suitability summarize karna.

- **False positive:** genuine activity galat tareeke se Suspicious mark ho jaye (analyst ka time waste).
- **False negative:** asli attack Normal mark ho jaye (attack miss ho sakta hai).
- Test set me 0 false positives mile; neeche ke 3 false negatives model ke **asli** misses hain.

```
print('False positive example:',
      'Legitimate late-night work on a new device is marked Suspici
print('False negative example:',
      'A disguised attack with few obvious indicators is marked Nor

# test set me asli misses nikaalte hain
test_check = X_test.copy()
test_check['actual'] = y_test.values
test_check['predicted'] = y_pred
fns = test_check[(test_check['actual'] == 'Suspicious') &
                 (test_check['predicted'] == 'Normal')]
print('Real false negatives in the test set:')
print(fns.to_string())

df.to_csv('Lab2_Login_ML_Result.csv', index=False)
print('Saved: Lab2_Login_ML_Result.csv')
```

```
False positive example: Legitimate late-night work on a new device i
False negative example: A disguised attack with few obvious indicato
Real false negatives in the test set:
    failed_logins  login_hour  new_device  download_mb      actual p
45            3.0           4           1        142.8  Suspicious
54            4.0          15           1        316.9  Suspicious
71            3.0          18           1        151.3  Suspicious
Saved: Lab2_Login_ML_Result.csv
```

# Lab file notes (copy karne ke liye)

**Q1:** Input columns — `failed_logins, login_hour, new_device, download_mb`; target — `label`. Cleaning se pehle: 1 missing `failed_logins`, 1 missing `download_mb`, 1 duplicate row; baad me: 0 missing, 0 duplicates, 100 rows.

**Q2:** 75 training / 25 testing rows (stratified, random_state=42). Training data par test karna misleading hai kyunki model wahi data yaad kar leta hai — unseen data par asli performance pata nahi chalti.

**Q3:** Accuracy **88.00%**; confusion matrix `[[19, 0], [3, 3]]` — 19 Normal sahi, 0 false alarms; 6 Suspicious me se 3 pakde, 3 miss (Normal mark hue). 100% se kam accuracy realistic hai kyunki genuine aur attack behaviour overlap karte hain (e.g. genuine user ke bhi failed logins ho sakte hain) — 100% aksar overfitting ya toy dataset ka sanket hota hai.

**Q4:** Cluster 0 (sab `new_device=1` wale logins) zyada suspicious lagta hai — usme 37 me se 13 Suspicious (35%) hain, jabki cluster 1 me 63 me se 12 (19%).

**Q5:** Sabse important features — `login_hour` (0.41), `failed_logins` (0.39), phir `download_mb` (0.19); `new_device` ko tree ne bilkul use nahi kiya (0.00). Naye records: (0 fails, subah 10 baje, purana device, 45 MB) → Normal; (8 fails, raat 2 baje, naya device) → Suspicious; (raat 11 baje naya device, 900 MB download) → Suspicious.

**Q6:** False positive — naya device par raat ko genuine kaam Suspicious mark ho jana (analyst ka time waste). False negative — record 54 (4 failed logins, dopahar 3 baje, naya device, 317 MB) asli me Suspicious tha par model ne Normal bola — chhupa hua attack miss ho gaya. Dataset ki limit: sirf 100 rows aur 4 numeric features — IP reputation, location jaise signals nahi hain. Rule: model jise Suspicious bole (aur high-download/naya-device wale 'Normal' cases) unhe action se pehle human analyst zaroor review kare.